# Reservas de salas de biblioteca

## Revisión por estaciones y material de apoyo para la entrega individual

6 de noviembre de 2026 | Prototipo académico Java de consola

Estado: P01-P11 superadas; 37 comprobaciones correctas. Se compiló el código guardado y se ejecutaron pruebas auxiliares sin modificar src.

Este documento es una base de revisión, no atribuye experiencias personales al estudiante. Personaliza la conclusión y añade tus propias capturas de consola si se exigen como evidencia individual.

## Contenido

1. Revisión de estaciones 1-8

2. Diagramas de flujo y clases

3. Matriz de trazabilidad P01-P11

4. Patrones, SOLID y antipatrones

5. Evidencias de ejecución

6. Reflexiones y conclusión

7. Instrucciones y estado de entrega

Fuentes: las 14 imágenes de la guía práctica proporcionadas y los archivos de src/biblioteca. No se consultaron fuentes externas. Material auxiliar elaborado con asistencia de Codex.

# Estación 1. Comprender el problema y modelar el flujo

## Evaluación

El flujo implementado es coherente: consulta, selección, construcción, suscripción de observadores, registro y confirmación. La estación exige además un diagrama y explicar responsabilidades; el código por sí solo no completa esta entrega.

## Prueba y resultado

Revisión conceptual: siga una solicitud para cuatro personas hasta Confirmada y una para diez personas hasta la salida sin reserva. No hay una prueba Java independiente para dibujar un flujo.

## Respuesta al punto de control

Main recoge entradas y configura dependencias; Strategy selecciona; Builder construye y valida; los repositorios consultan y almacenan; Facade coordina; State autoriza transiciones; Observer comunica los cambios. Una clase que hiciera todo tendría baja cohesión porque mezclaría motivos distintos de cambio.

# Estación 2. Crear el dominio

## Evaluación

Correcto. Estudiante y Sala validan entradas, mantienen atributos privados y finales, y exponen getters. Las salas tienen capacidades 2, 4, 6 y 8.

## Prueba y resultado

Mostrar las cuatro salas, comprobar capacidades y nombre del estudiante; rechazar identificador vacío y capacidad cero. Resultado: ocho comprobaciones correctas.

## Respuesta al punto de control

private protege el acceso y final impide reasignar los campos tras construir el objeto. En estas clases, los campos son String e int y no hay setters, lo que mantiene estables los datos. final por sí solo no vuelve inmutable cualquier objeto referenciado.

Código ejecutable: entregables/pruebas/estacion2/Main.java. La guía Markdown contiene el código completo de cada estación. Los auxiliares son solo de prueba y no sustituyen las clases de la guía.

# Estación 3. Construir reservas con Builder

## Evaluación

Correcto. ReservaBuilder centraliza datos obligatorios, fecha no pasada, duración y personas positivas, y capacidad suficiente. Genera el código y crea una reserva Pendiente. La dependencia de State es acumulativa, tal como advierte la guía.

## Prueba y resultado

P01: construir una reserva válida; P02: omitir estudiante. Añadidos: capacidad insuficiente, fecha pasada, duración cero y personas cero. Resultado: seis comprobaciones correctas.

## Respuesta al punto de control

Estas validaciones aseguran que la reserva nazca válida y deben permanecer en Builder, evitando repetirlas en Main. Los métodos encadenados hacen explícito el significado de cada dato y reducen errores de orden de argumentos.

Código ejecutable: entregables/pruebas/estacion3/Main.java. La guía Markdown contiene el código completo de cada estación. Los auxiliares son solo de prueba y no sustituyen las clases de la guía.

# Estación 4. Seleccionar salas con Strategy

## Evaluación

Correcto. SeleccionPorCapacidad filtra y elige el mínimo suficiente; SeleccionSalaPreferida busca el código sin distinguir mayúsculas y comprueba capacidad. Ambas devuelven Optional.

## Prueba y resultado

P03: cuatro personas por capacidad -> B. P04: preferida C para cuatro -> C. P05: preferida B para siete -> vacío. También se comprobó que para tres personas el mínimo suficiente es B. Cuatro comprobaciones correctas.

## Respuesta al punto de control

Una tercera estrategia podría elegir la sala de mayor capacidad: bastaría crear otra clase que implemente EstrategiaSeleccionSala y elegirla al configurar Main; no se cambia Facade ni las estrategias existentes. Para seleccionar por equipamiento también habría que modelar ese dato en Sala o consultarlo: el modelo actual no lo contiene.

Código ejecutable: entregables/pruebas/estacion4/Main.java. La guía Markdown contiene el código completo de cada estación. Los auxiliares son solo de prueba y no sustituyen las clases de la guía.

# Estación 5. Acceso a datos y Facade

## Evaluación

Correcto en su versión final. Los repositorios separan contratos de implementaciones en memoria. Facade recibe dependencias, selecciona, construye, suscribe, registra y confirma. El constructor de tres argumentos es la ampliación de la estación 6.

## Prueba y resultado

P06: comprobar que la reserva retornada es la misma almacenada y está Confirmada. P10: solicitar diez personas y comprobar Optional vacío sin aumentar el repositorio. Dos comprobaciones correctas.

## Respuesta al punto de control

Main hace una operación de alto nivel y no repite el orden interno del caso de uso. Para sustituir memoria por una fuente real se implementa RepositorioSalas o RepositorioReservas y se inyecta en Main; la coordinación de Facade permanece igual si se conserva el contrato.

Código ejecutable: entregables/pruebas/estacion5/Main.java. La guía Markdown contiene el código completo de cada estación. Los auxiliares son solo de prueba y no sustituyen las clases de la guía.

# Estación 6. Notificar con Observer

## Evaluación

Correcto. Reserva guarda suscriptores por interfaz, permite agregarlos y retirarlos, y notifica desde cambiarEstado. Facade los conecta antes de confirmar. List.copyOf evita iterar directamente sobre una lista que un suscriptor pudiera modificar.

## Prueba y resultado

P07: ambos observadores reciben confirmación. P08: cancelación válida notifica. P09: confirmar cancelada se rechaza sin emitir avisos. Además se retira un observador y se comprueba que no recibe eventos. Cinco comprobaciones; los contadores son auxiliares de prueba y se ejecutan junto con las dos clases reales.

## Respuesta al punto de control

Reserva conoce ObservadorReserva, no la forma de notificar ni registrar. Un nuevo observador puede añadirse sin cambiar Reserva. El texto actual concatena "su reserva" con el código sin espacio; es una diferencia visual respecto a la guía y no afecta al patrón.

Código ejecutable: entregables/pruebas/estacion6/Main.java. La guía Markdown contiene el código completo de cada estación. Los auxiliares son solo de prueba y no sustituyen las clases de la guía.

# Estación 7. Controlar el ciclo de vida con State

## Evaluación

Correcto. Pendiente permite confirmar o cancelar; Confirmada permite cancelar o finalizar; Cancelada y Finalizada rechazan todas las operaciones. Los rechazos lanzan excepción antes de cambiarEstado, por lo que no notifican.

## Prueba y resultado

Se prueban las cuatro transiciones válidas y las ocho combinaciones inválidas entre cuatro estados y tres operaciones, repartidas entre las estaciones 6 y 7. P11 comprueba finalizar una confirmada y recibir evento. Once comprobaciones en esta estación.

## Respuesta al punto de control

Frente a una Reserva llena de if/else, cada estado concentra sus reglas. Agregar un estado requiere su clase y actualizar las transiciones que conduzcan a él; no significa que ningún archivo existente vaya a cambiar.

Código ejecutable: entregables/pruebas/estacion7/Main.java. La guía Markdown contiene el código completo de cada estación. Los auxiliares son solo de prueba y no sustituyen las clases de la guía.

# Estación 8. Integrar la aplicación

## Evaluación

Correcto. El Main original compila y termina sin excepciones no controladas. Configura dependencias y presenta los escenarios de la guía. Los identificadores de Camila y Samuel son E004 y E005 en lugar de E003 y E004; es una diferencia de datos de ejemplo sin efecto funcional.

## Prueba y resultado

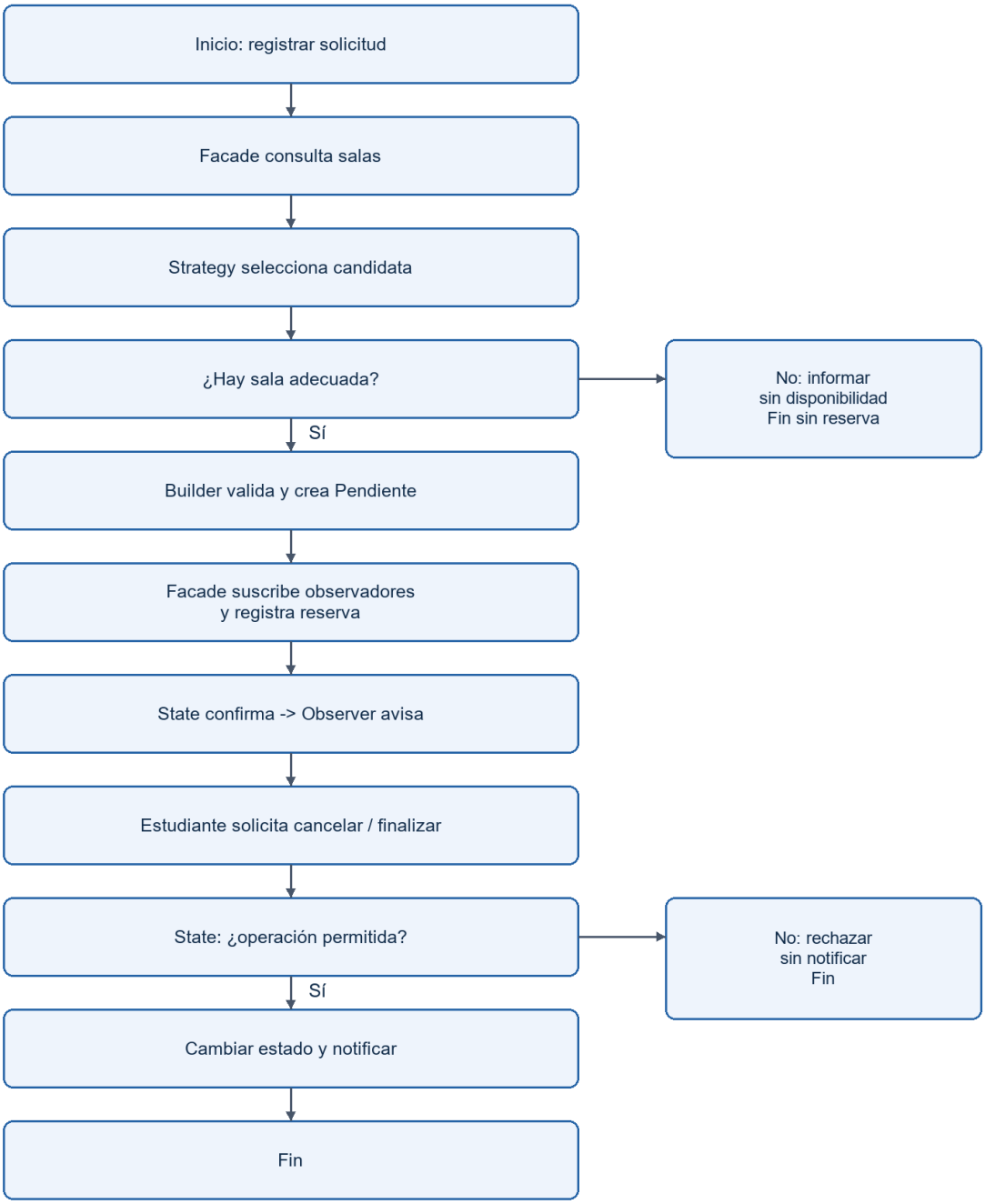
Ejecutar el Main original: Laura B confirmada y luego cancelada; rechazo de confirmar cancelada; Andrés C; Camila sin sala; Samuel A finalizada. Una comprobación de terminación más la revisión de la salida, guardada como evidencia.

## Respuesta al punto de control

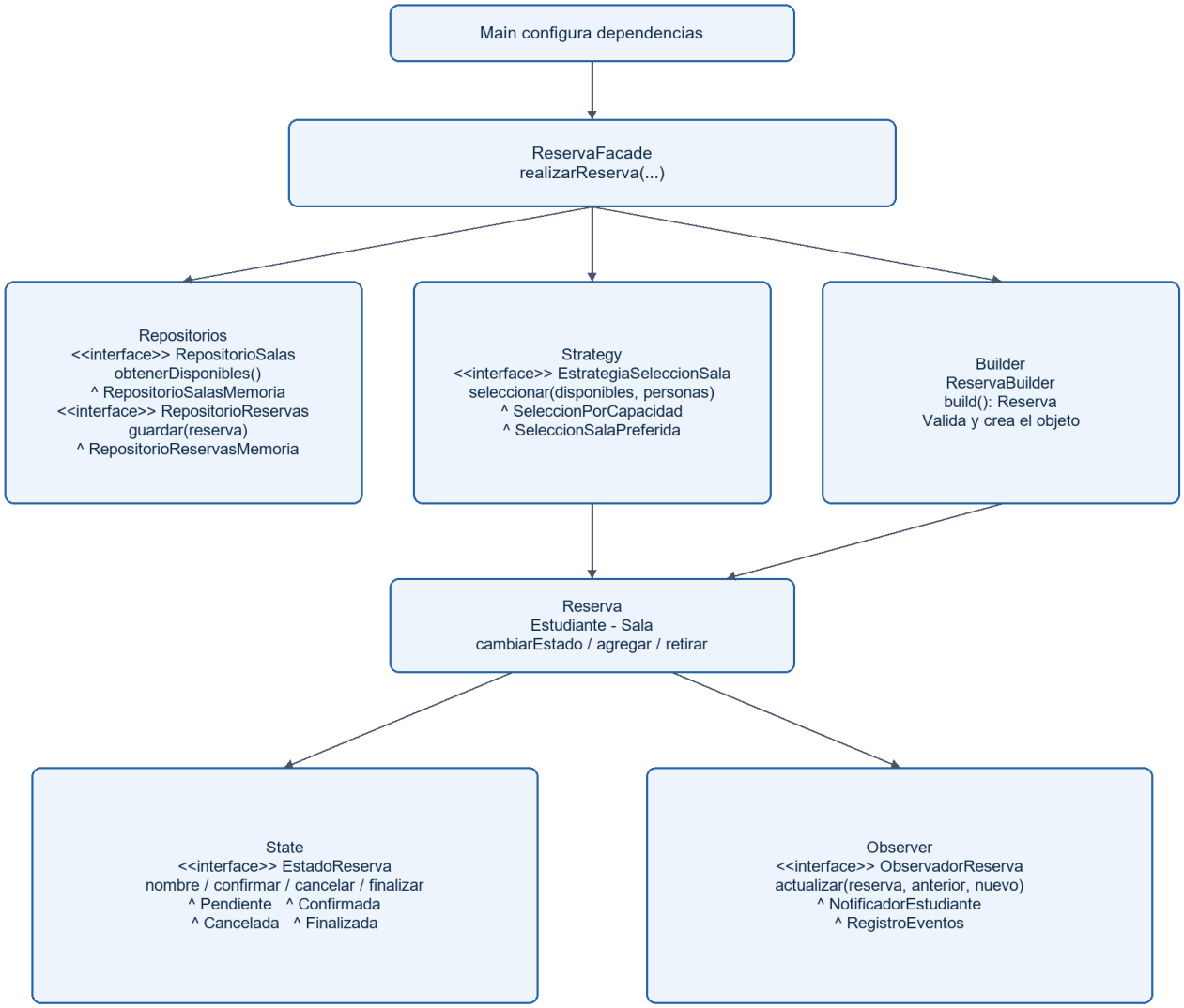
El Main original no cubre por sí solo P02 ni P05. Las pruebas auxiliares completan la matriz P01-P11. La lista fija simula disponibilidad: no detecta cruces horarios ni persiste entre ejecuciones, y una sala puede reaparecer en varias reservas.

Código ejecutable: entregables/pruebas/estacion8/Main.java. La guía Markdown contiene el código completo de cada estación. Los auxiliares son solo de prueba y no sustituyen las clases de la guía.

# Diagrama de flujo



# Diagrama de clases: vista general




^ implementa la interfaz; flecha: uso/dependencia. Fuentes Mermaid: relaciones detalladas.

Las fuentes Mermaid editables detallan las relaciones, incluidas las asociaciones de Reserva con Estudiante, Sala, estados y observadores. No se modifica el estado desde Main; se invocan operaciones del dominio.

# Matriz de trazabilidad

| ID | Escenario y esperado | Clases / patrón | Resultado |
|---|---|---|---|
| P01 | Construcción completa<br>Reserva válida, Pendiente | ReservaBuilder, Reserva<br>Builder \| Est. 3 | OK |
| P02 | Sin estudiante<br>Rechazo con mensaje | ReservaBuilder<br>Builder \| Est. 3 | OK |
| P03 | Capacidad para 4<br>Sala B | SeleccionPorCapacidad<br>Strategy \| Est. 4 | OK |
| P04 | Preferida C para 4<br>Sala C | SeleccionSalaPreferida<br>Strategy \| Est. 4 | OK |
| P05 | Preferida B para 7<br>Optional vacío | SeleccionSalaPreferida<br>Strategy \| Est. 4 | OK |
| P06 | Reserva exitosa<br>Registrada y Confirmada | ReservaFacade, RepositorioReservasMemoria<br>Facade \| Est. 5 | OK |
| P07 | Cambio a Confirmada<br>Ambos observadores avisados | Reserva, Pendiente, NotificadorEstudiante, RegistroEventos<br>Observer + State \| Est. 6 | OK |
| P08 | Cancelar Confirmada<br>Cancelada y evento | Confirmada, Cancelada, Reserva<br>State + Observer \| Est. 6 | OK |
| P09 | Confirmar Cancelada<br>Rechazo, sin evento | Cancelada, Reserva<br>State \| Est. 6 | OK |
| P10 | Solicitud para 10<br>No crea ni registra | SeleccionPorCapacidad, ReservaFacade<br>Strategy + Facade \| Est. 5 | OK |
| P11 | Finalizar Confirmada<br>Finalizada y evento | Confirmada, Finalizada, Reserva<br>State + Observer \| Est. 7 | OK |

# Los cinco patrones

## Builder

ReservaBuilder reúne parámetros y valida antes de crear Reserva.

## Strategy

EstrategiaSeleccionSala permite intercambiar selección por capacidad o preferencia.

## Facade

ReservaFacade coordina el caso de uso mediante una operación de alto nivel.

## Observer

Reserva publica cambios a ObservadorReserva; NotificadorEstudiante y RegistroEventos reaccionan.

## State

EstadoReserva y sus cuatro implementaciones autorizan o rechazan operaciones según el estado actual.

# SOLID y antipatrones

## SRP

Builder construye; Strategy selecciona; State controla transiciones; Observer notifica; repositorios almacenan; Facade coordina. Evita una clase que concentre todo.

## OCP

Se pueden añadir estrategias y observadores implementando sus interfaces. No hace falta agregar un switch en Facade por cada algoritmo.

## LSP

Las estrategias devuelven una sala válida o Optional vacío según el contrato. Los estados implementan todas las operaciones; rechazar una transición es parte del contrato de State, no una promesa de que todas estén permitidas.

## ISP

Las interfaces de estrategia, observador y repositorios son pequeñas y específicas; no obligan a implementar operaciones ajenas.

## DIP

Facade depende de interfaces de repositorio y estrategia; Reserva depende de interfaces de estado y observador. Main selecciona las implementaciones concretas.

## Antipatrones evitados

No se concentra todo en Main ni en Reserva. No hay un switch de selección ni estados representados por cadenas modificables. No se instancian los notificadores concretos dentro de Reserva. Las excepciones se presentan en consola en lugar de ignorarse.

Comprobar capacidad en Strategy selecciona candidatas; validarla en Builder impide construir reservas inválidas. Son responsabilidades diferentes. No se añadieron funcionalidades fuera del alcance.

# Evidencias: construcción y estrategias

```
SALIDA REAL DE EJECUCION - REPRESENTACION DE TEXTO
No es una captura del IDE. 06/11/2026

=== ESTACION 3: BUILDER ===
Reserva R-155F2550 | Laura | 2026-11-13 14:00 | Sala B (B, capacidad: 4) | Estado:Pendiente
OK: P01: construccion valida, estado inicial Pendiente
OK: P02: sin estudiante | Faltan datos obligatorios para la reserva.
OK: Capacidad insuficiente | La sala no tiene la capacidad suficiente.
OK: Fecha pasada | Fecha, duracion o cantidad de personas invalida.
OK: Duracion cero | Fecha, duracion o cantidad de personas invalida.
OK: Personas cero | Fecha, duracion o cantidad de personas invalida.
```

```
SALIDA REAL DE EJECUCION - REPRESENTACION DE TEXTO
No es una captura del IDE. 06/11/2026

=== ESTACION 4: STRATEGY ===
OK: P03: capacidad para 4 -> B
OK: P04: preferida C para 4 -> C
OK: P05: preferida B para 7 -> vacio
OK: Elige la menor capacidad suficiente
```

```
SALIDA REAL DE EJECUCION - REPRESENTACION DE TEXTO
No es una captura del IDE. 06/11/2026

=== ESTACION 5: FACADE ===
OK: P06: reserva registrada y confirmada
No hay una sala disponible que cumpla la solicitud.
OK: P10: para 10 personas no crea ni registra otra reserva
```

Imágenes generadas a partir de la salida real guardada en resultados.txt. Son representaciones de texto, no capturas de pantalla de un IDE.

# Evidencias: transiciones y notificaciones

```
SALIDA REAL DE EJECUCION - REPRESENTACION DE TEXTO
No es una captura del IDE. 06/11/2026

=== ESTACION 6: OBSERVER ===
Notificacion a Laura: su reservaR-6478EE31 cambio de Pendiente a Confirmada.
Evento registrado: R-6478EE31 cambio de Pendiente a Confirmada.
Observador 1: Pendiente -> Confirmada
Observador 2: Pendiente -> Confirmada
OK: P07: ambos suscriptores reciben la confirmacion
Notificacion a Laura: su reservaR-6478EE31 cambio de Confirmada a Cancelada.
Evento registrado: R-6478EE31 cambio de Confirmada a Cancelada.
Observador 1: Confirmada -> Cancelada
Observador 2: Confirmada -> Cancelada
OK: P08: cancelar confirmada cambia estado y notifica
OK: P09: confirmar cancelada se rechaza | No se puede confirmar una reserva cancelada.
OK: P09: rechazo no genera notificaciones
OK: Observador retirado no recibe avisos
```

```
SALIDA REAL DE EJECUCION - REPRESENTACION DE TEXTO
No es una captura del IDE. 06/11/2026

=== ESTACION 7: STATE ===
OK: No se finaliza una Pendiente | No se puede finalizar una reserva pendiente.
OK: Rechazo conserva estado
OK: No se confirma dos veces | La reserva ya esta confirmada.
OK: P11: finalizar confirmada cambia estado y notifica
OK: No se cancela una Finalizada | No se puede cancelar una reserva finalizada
OK: No se confirma una Finalizada | No se puede confirmar una reserva finalizada
OK: No se finaliza dos veces | Esta reserva ya esta finalizada
OK: Estado terminal conserva estado y no notifica rechazos
OK: Pendiente -> Cancelada
OK: No se cancela dos veces | La reserva ya esta cancelada.
OK: No se finaliza una Cancelada | No se puede finalizar una reserva cancelada
```

P09 comprueba explícitamente que el contador de eventos no cambia al rechazar la operación. P11 comprueba tanto Finalizada como la emisión de evento.

# Reflexiones: respuestas base

## 14. Responsabilidades separadas

Se separaron consulta, selección, construcción, persistencia, coordinación, estados y notificación. Una única clase acumularía cambios por motivos distintos y dificultaría probar cada parte.

## 15. Ventaja de Builder

Los nombres de sus métodos explican cada dato y build centraliza las validaciones; un constructor con numerosos argumentos facilita confundir posiciones.

## 16. Extensión de Strategy

Se crea una implementación de EstrategiaSeleccionSala y se pasa a realizarReserva. Facade invoca el contrato sin conocer el algoritmo concreto.

## 17. Complejidad oculta por Facade

Oculta consulta, selección, construcción, suscripción, registro y confirmación. Main no depende del orden de esos pasos.

## 18. Desacoplamiento de Observer

Reserva invoca actualizar sobre ObservadorReserva. Los suscriptores deciden cómo notificar o registrar sin introducir sus detalles dentro de Reserva.

# Reflexiones: continuación

## 19. Aporte de State

Un String solo guarda una etiqueta; State reúne las operaciones permitidas y los rechazos de cada estado y evita modificar libremente la etiqueta desde Main.

## 20. Dos principios SOLID

SRP: ReservaBuilder valida y construye mientras RegistroEventos registra cambios. DIP: ReservaFacade recibe RepositorioSalas y RepositorioReservas por sus interfaces.

## 21. Antipatrón evitado

God Object: Main configura y ejecuta ejemplos, pero no contiene las reglas de selección, validación ni transición. Esas reglas están en sus componentes.

## 22. Disponibilidad simulada

RepositorioSalasMemoria siempre devuelve la misma lista; no considera fecha ni hora ni bloquea salas. Para una agenda real habría que ampliar el contrato de consulta con el intervalo e implementar un repositorio que detecte solapamientos y persista reservas.

# Conclusión técnica para personalizar

Borrador para personalizar: La separación mediante Builder, Strategy, Facade, Observer y State permite localizar las reglas y probarlas por componente. Una decisión central es inyectar repositorios y observadores desde Main, manteniendo las abstracciones en el dominio. La ejecución revisada superó P01-P11 y 37 comprobaciones. La integración entre cambios de estado y notificaciones exige conectar los observadores antes de confirmar. Como mejora futura propondría disponibilidad por intervalo y persistencia, con una consulta que considere fecha, hora y duración. Este prototipo es de consola, usa una lista fija, no controla solapamientos ni conserva datos al terminar. Antes de entregar, añade las dificultades que tú experimentaste y explica cómo las resolviste; este texto no afirma experiencias personales tuyas.

## Diferencias menores

Hay variaciones de tildes, espacios, puntos finales, sangría e identificadores de ejemplo. No afectan los resultados funcionales. Se conservaron las clases originales.

## Cómo reproducir

Ejecuta entregables/ejecutar_pruebas.ps1 desde PowerShell. Requiere JDK 17 o superior. Compila src y MainPruebas en una carpeta temporal y guarda la salida en evidencias/resultados.txt. La última línea esperada es TOTAL: 37 comprobaciones correctas.

## Pruebas desde Main

Cada carpeta pruebas/estacion2 a estacion8 contiene un Main.java completo listo para copiar temporalmente en el Main del proyecto. Respalda el original, usa una sola versión a la vez y restaura al terminar. La estación 1 requiere revisión del diagrama, no un Main adicional.

## Estado de los entregables

Se incluyen proyecto Java comprimido, PDF, guía editable, diagramas PNG y Mermaid, matriz CSV, código de pruebas y salida real. Pendiente del estudiante: personalizar la reflexión individual y, si corresponde, capturar la ejecución en su propio IDE.